

PROP PRIMERA ENTREGA

Versió 1.1

Grup 13.2 - Alicia Ageno

BARANDA DOMINGUEZ, JORDI - jordi.baranda@estudiantat.upc.edu

CLUSELLAS CÁNOVAS, JUAN - juan.clusellas@estudiantat.upc.edu

SOLANES SERRAT, FERRAN - ferran.solanes@estudiantat.upc.edu

TURU ROCA, MARC - marc.turu@estudiantat.upc.edu



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



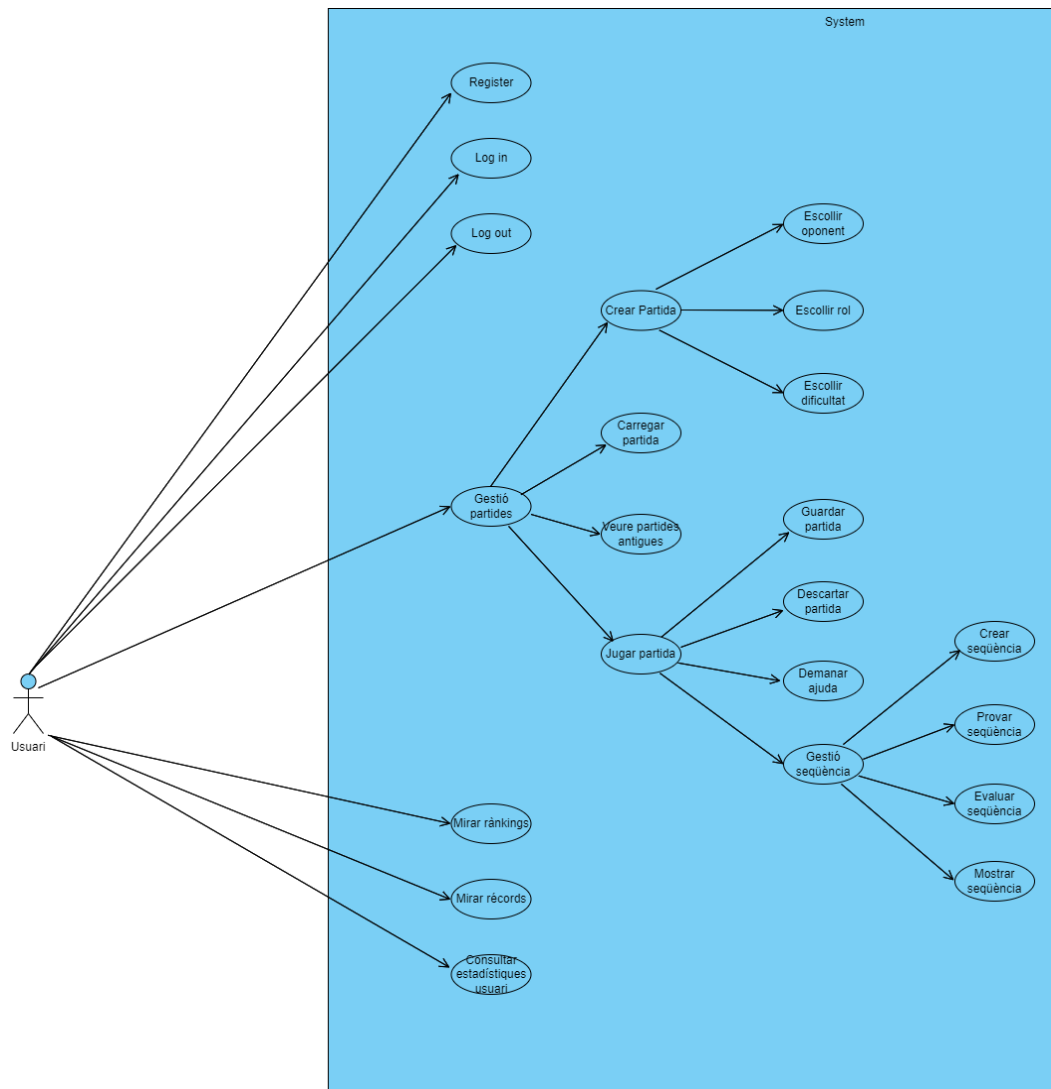
1. Casos d'ús	2
1.1 Diagrama de casos d'ús	2
1.2. Descripció casos d'ús	3
2. Diagrama del model conceptual	9
2.1 Disseny del diagrama del model conceptual	9
2.2 Descripció de les classes	10
2.2.1 User	10
2.2.2 User_persona	11
2.2.3 User_maquina	12
2.2.4 Partida	12
2.2.5 Ronda	14
2.2.6 Seqüència	14
2.2.7 Seqüència Intentada	15
2.2.8 Seqüència Solució	16
2.2.9 Seqüència Verificació	16
2.2.10 Rànk	16
2.2.11 Rànk fàcil	17
2.2.12 Rànk_normal	17
2.2.13 Rànk_difícil	17
2.2.14 Rànk_pvp	18
2.2.15 Record	18
2.2.16 Type_user	18
2.2.17 dificultats	19
2.2.18 colors	19
2.2.19 type_seq	20
2.2.20 Pair	20
3. Estructures de dades i algorismes utilitzats	21
3.1 Estructures de dades	21
3.2 Algorismes	22
Five Guess Algorithm	22
4. Relació de les classes implementades per membre de l'equip	24

1. Casos d'ús

En aquest primer apartat es comenten els casos d'ús que es donaran dins del nostre desenvolupament del joc, servint com a representació de les funcionalitats que l'actor usuari podrà dur a terme dins del sistema.

Es mostra l'estructura d'aquests, quina relació tenen entre ells en el diagrama de casos d'ús i una breu explicació de cadascun.

1.1 Diagrama de casos d'ús



1.2. Descripció casos d'ús

Cas d'ús 1 - Register

Actor: Usuari

Comportament:

1. L'usuari introdueix un nou nom d'usuari i una nova contrasenya
2. El sistema confirma aquests camps
- 3 El sistema crea el nou perfil amb els paràmetres indicats.

Errors possibles i cursos alternatius: Ja existeix un usuari amb aquell nom d'usuari, es cancel·la el registre i s'informa de l'error.

Cas d'ús 2 - Log in

Actor: Usuari

Comportament:

1. L'usuari introdueix el seu nom d'usuari i la seva contraseña
2. El sistema comprova el valors per veure si coincideixen.
3. El sistema procedeix a confirmar l'entrada de l'usuari en cas de que els valors siguin coincidents amb algun usuari ja registrat.

Errors possibles i cursos alternatius: Si l'usuari no existeix, el sistema informa d'error.
Si la contrasenya no coincideix amb la registrada, el sistema informa d'error

Cas d'ús 3 - Log out (No implementat encara)

Actor: Usuari

Comportament:

1. L'usuari, que prèviament ha iniciat sessió pel seu perfil, decideix tancar-la
2. El sistema li mostra una opció per a fer-ho i tanca la sessió.

Errors possibles i cursos alternatius:

Cas d'ús 4 - Crear Partida

Actor: Usuari

Comportament:

1. L'usuari dona d'alta una nova partida
2. El sistema crea una nova partida amb els paràmetres de tipus escollits i li assigna un identificador únic.

Errors possibles i cursos alternatius: Es comprova que l'usuari no té cap partida en curs. Si és així, es pregunta si es vol eliminar aquesta i crear-ne una de nova o, en cas contrari, no fer res.

Si algun dels dos usuaris supera el límit de partides no acabades, es mostra un error.

Cas d'ús 5 - Escollir oponent

Actor: Usuari

Comportament:

1. L'usuari escull entre jugar contra la màquina o contra una altre persona.

Errors possibles i cursos alternatius: En cas d'escollir jugar contra un altre jugador, el sistema demana el login d'aquest jugador i es comença el cas d'ús de login.

Cas d'ús 6 - Escollir rol

Actor: Usuari

Comportament:

1. L'usuari escollix entre ser Codemaker o Codebreaker.

2. S'assignen els camps de codemaker i codebreaker als dos usuaris que participen a la partida.

Errors possibles i cursos alternatius: En cas de ser partida jugador contra jugador només escull rol el jugador que ha iniciat la partida.

Cas d'ús 7 - Escollir dificultat

Actor: Usuari

Comportament:

1. L'usuari escull la dificultat de la partida entre fàcil, normal i difícil. Aquesta opció només es presenta quan es juga contra la màquina.

2. S'assigna aquesta dificultat a la partida.

Errors possibles i cursos alternatius:

Cas d'ús 8 - Gestió de partida

Actor: Usuari

Comportament:

1. Permet crear, carregar, descartar, guardar una partida.

Errors possibles i cursos alternatius:

Cas d'ús 9 - Carregar partida (Entrega 3)

Actor: Usuari

Comportament:

1. L'usuari escull quina partida vol carregar de totes les que té en curs
2. Es carrega la partida per a que la continui jugant l'usuari.
2. Aquesta partida s'esborra de les que estan en curs.

Errors possibles i cursos alternatius: Si l'usuari no té partides en curs, el sistema informa d'error.

Cas d'ús 10 - Descartar partida

Actor: Usuari

Comportament:

1. L'usuari decideix acabar la partida en meitat d'una o quan ja ha acabat.
2. La partida passa a formar part de les partides acabades de l'usuari en qüestió.

Errors possibles i cursos alternatius:

Cas d'ús 11 - Guardar partida (Entrega 3)

Actor: Usuari

Comportament:

1. L'usuari escull guardar una partida no acabada per tornar a jugarla en el futur.
2. El sistema comprova que no es superi el limit de partides guardades. En cas de que no passi, la guarda.

Errors possibles i cursos alternatius: Si l'usuari té 5 partides guardades, el sistema informa d'error.

Cas d'ús 12 - Mirar rànkings

Actor: Usuari

Comportament:

1. L'usuari accedeix i visualitza la classificació general de jugadors, on es poden filtrar els rankings segons dificultat i tipus d'oponent (opcions: fàcil, normal, difícil, vs player).
2. En cas que no hi hagi cap partida jugada, el rànkings apareix buit.

Errors possibles i cursos alternatius: Com és una consultora no hi ha errors.

Cas d'ús 13 - Mirar rècords

Actor: Usuari

Comportament:

1. L'usuari accedeix a veure quin usuari té un rècord (aquí no es té en compte la dificultat o el contrincant).
2. En cas que no s'hagi batut cap record apareix buit.

Errors possibles i cursos alternatius: Al ser una consultora no té possibles errors.

Cas d'ús 14 - Veure partides antigues

Actor: Usuari

Comportament:

1. L'usuari accedeix al tauler de partides anteriors acabades.
2. Es mostra els diferents intents de cada ronda i la seqüència de verificació retornada.

Errors possibles i cursos alternatius: En cas que la partida que es vol veure no estigui acabada, el sistema informa d'error.

Cas d'ús 15 - Gestió de seqüència

Actor: Usuari

Comportament: Permet crear, evaluar, mostrar i provar seqüències.

Errors possibles i cursos alternatius:

Cas d'ús 16 - Crear seqüència

Actor: Usuari

Comportament:

1. L'usuari és codemaker i crea una seqüència del codi.

Errors possibles i cursos alternatius: Si la seqüència no està completa, el sistema informa d'error.

Cas d'ús 17 - Evaluar seqüència

Actor: Usuari

Comportament:

1. L'usuari és codemaker i evalua la seqüència del contrincant, posant les espigues corresponents.

Errors possibles i cursos alternatius: Si l'usuari no evalua bé la seqüència del rival, no deixa avançar fins que estigui bé.

Cas d'ús 18 - Prova seqüència

Actor: Usuari

Comportament:

1. L'usuari és codebreaker i fa un intent de'endevinar la seqüència.

Errors possibles i cursos alternatius: Si la seqüència no està completa, es mostra error..

Cas d'ús 19 - Mostrar seqüència

Actor: Usuari

Comportament:

1.1 Si l'usuari es codemaker, pot demanar tornar a veure la seqüència creada en cas de que no s'enrecordi.

1.1 El sistema mostra la seqüència al codemaker.

1.2 Si l'usuari codebreaker, se li mostra la seqüència a desxifrar al final de la partida.

Errors possibles i cursos alternatius: El codebreaker demana veure la seqüència sense que hagi acabat la partida i no se li permet.

Cas d'ús 20 - Demanar ajuda

Actor: Usuari

Comportament:

1. L'usuari demana ajuda en un cert moment de la partida.

2. Se li dona un hint per a resoldre la seqüència creada pel codemaker.

3. Es penalitza a l'usuari.

Errors possibles i cursos alternatius: En cas que l'usuari ja hagi demanat ajuda, el sistema informa d'error.

Cas d'ús 21 - Consultar estadístiques usuari

Actor: Usuari

Comportament:

1. L'usuari demana consultar les seves pròpies estadístiques (com ara les partides jugades, les seves puntuacions, les partides guanyades, etc).

2. En cas de que l'usuari no hagi jugat cap partida, se li mostren les estadístiques a 0/buides.

Errors possibles i cursos alternatius: Si l'usuari és de tipus màquina, es mostra un error.

Cas d'ús 22 - Jugar partida

Actor: Usuari

Comportament:

1. L'usuari recorre un bucle on es recorren les diferents rondes de la partida.

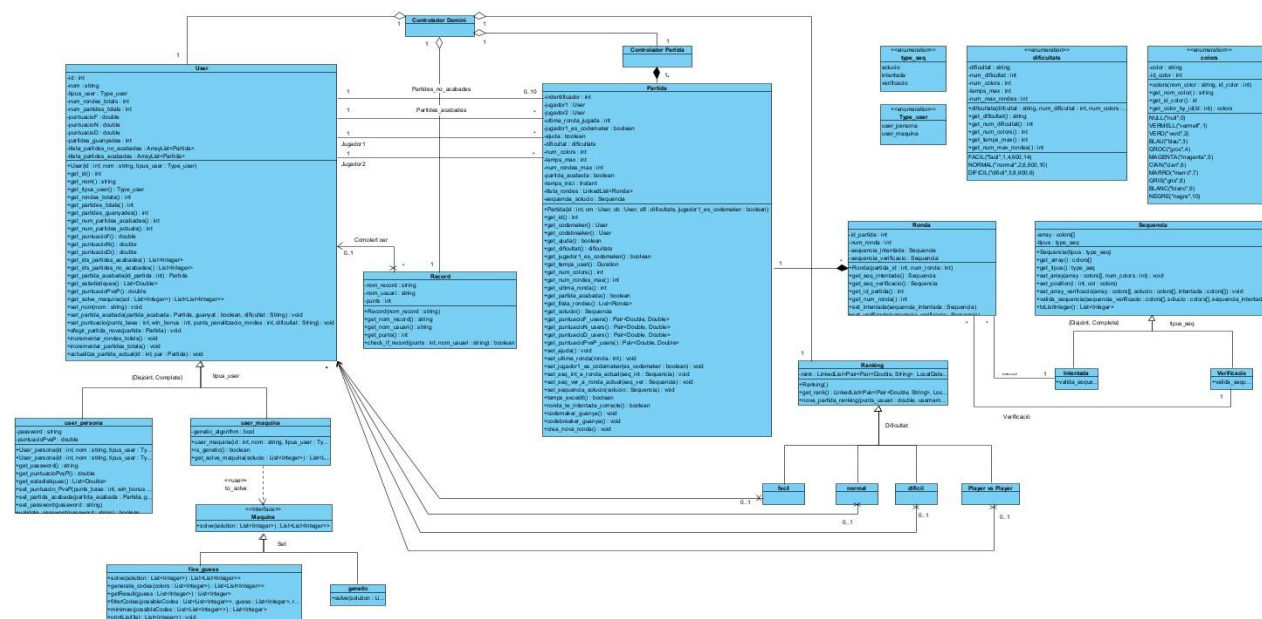
Aquest bucle para en cas que s'acabin les rondes, es guanyi la partida o l'usuari decideixi guardar/descartar la partida.

En cada iteració el codebraker prova una seqüència i el codemaker li evalua.

Errors possibles i cursos alternatius:

En aquesta segona secció, es mostra el diagrama UML on es veuen respresentades totes les classes que són necessàries pel desenvolupament del joc, els seus atributs, mètodes, i la seva corresponent relació entre elles.

2.1 Disseny del diagrama del model conceptual



Restriccions Textuals:

RT1: Claus externas: (*User*, id), (*user_persona*, id), (*user_maquina*, id), (*Record*, nom_record), (*Partida*, id), (*Ronda*, num_ronda + Partida::identificador)

RT2: El jugador1 d'una partida no pot ser de tipus maquina.

RT3: El num_ronda d'una ronda en concret no pot ser més gran que el nombre màxim de rondes a jugar segons la dificultat d'aquella partida.

RT4: L'última_ronda_jugada d'una partida no pot ser més gran que el nombre màxim de rondes a jugar segons la dificultat d'aquella aptrida.

RT5: Una partida no pot ser alhora acabada i no acabada.

2.2 Descripció de les classes

Aquesta descripció de les classes es farà sobre totes les que apareixen al diagrama UML i es comentaran tots els enumerations usats i que apareixen en el mateix.

2.2.1 User

Aquesta classe fa referència als usuaris que jugaran al joc. Aquest usuaris poden ser de tipus persona o de tipus màquina (`user_persona` i `user_maquina`). Els users tenen un id únic que els identifica. A part, té altres atributs com `nom`, `tipus_user`, i altres que serviran per desenvolupar diferents funcionalitats del mateix, com les puntuacions d'aquest.

En aquesta superclasse estan implementats la majoria de getters i setters que permeten obtenir els valors d'aquests atributs mencionats.

Atributs

- **protected int id:** Identificador de l'usuari
- **protected String nom:** Nom amb el que es registrarà l'usuari i amb el qual apareixerà a l'interfície del joc
- **protected Type_user tipus_user:** Enumeration que diferencia els dos tipus d'usuari
- **protected int num_rondes_totals:** Número de rondes totals que ha jugat l'usuari des de que està registrat
- **protected int num_partides_totals:** Número de partides totals que ha jugat l'usuari al joc
- **protected double puntuacioF:** Puntuació de l'usuari de només les partides que ha jugat de tipus fàcil.
- **protected double puntuacioN:** Puntuació de l'usuari de només les partides que ha jugat de tipus normal.
- **protected double puntuacioD:** Puntuació de l'usuari de només les partides que ha jugat de tipus difícil.
- **protected int partides_guanyades:** Número de partides que ha guanyat l'usuari dins de totes les que ha jugat.
- **protected List<Partida> llista_partides_no_acabades:** Array llista de tipus Partida que conté totes les partides que l'usuari no ha acabat (no ha ni guanyat ni perdut)
- **protected List<Partida> llista_partides_acabades:** Array llista de tipus Partida que conté totes les partides que l'usuari ha acabat (ha guanyat o perdut)

Mètodes principals

- **public void set_partida_acabada(Partida partida_acabada, boolean gunyat, String diifcultat):** Funció que serveix per a realitzar modificacions sobre l'usuari i els seus atributs després de la finalització d'una partida. Algunes d'aquestes modificacions són simplement per a actualitzar atributs d'una forma simple i les altres per cridar a altres funcions com...
- **public void set_puntuacio(int punts_base, int win_bonus, int punts_penlaitzacio_rondes, String dificultat):** Aquesta funció, cridada quan s'acaba una partida i s'han de restablir les puntuacions, actualitza els valors de les puntuacions segons el tipus de dificultat que li arriba com a paràmetre perquè aquest, posteriorment, pugui ser tingut en compte per futures actualitzacions de rànkings i possibles rècords.

2.2.2 User_persona

Aquesta subclasse de User, ens permet implementar funcionalitats extres de User que només poden ser usades/provades per aquest tipus d'usuari.

Bàsicament ens serveix per diferenciar el tipus de partides que juga cada user. Aquest user_persona podrà jugar partides PvsP i obtenir puntuacions d'aquest tipus.

A part, aquest tipus d'usuari podrà tenir contrasenya per a poder fer el registre i el log in per a identificar-se.

Atributs

- **Tots els heredats de la superclasse User**
- **private String password:** Contrassenya que utilitzarà l'usuari per registrar-se i, posteriorment, fer el seu log in
- **private double puntuacioPvsP:** Puntuació de l'usuari de només les partides que ha jugat de tipus PvsP

Mètodes principals

- **public vector<Double> get_estadistiques():** Aquesta funció permet retornar i mostar les estadístiques d'un usuari de tipus persona. D'aquesta manera, i retornant-les amb un vector, es pot accedir d'una manera senzilla als valors dels atributs del usuari que poden ser útils per fer comparacions entre ells o per, simplement, mostrar-los per pantalla.

- **public void set_puntuacio_PvsP(int punts_base, int win_bonus, int punts_penalitzacio_rondes):** Igual que a la superclasse però amb la puntuació de tipus PvsP

2.2.3 User_maquina

Aquesta subclasse de User, ens permet implementar funcionalitats extres de User que només poden ser usades/provades per aquest tipus d'usuari.

Bàsicament ens serveix per saber quin tipus d'algorisme s'utilitzarà per a la resolució del joc i, en conseqüència, quin tipus de màquina és.

Atributs

- **Tots els heredats de la superclasse User**
- **private boolean genetic_algorithm:** Booleà que serveix per diferenciar quin tipus d'algorisme es farà servir i quin tipus de màquina és l'usuari2

Mètodes principals

- **List<List<Integer>> get_solve_maquina(List<Integer> solucio):** És, bàsicament, la funció que crida a l'algorisme per resoldre el joc. Encara que sigui només una funció que crida a una altra, l'algorisme consta de tanta complexitat, que val la pena mencionar-la.

Aquesta interfície màquina, implementa els dos algorismes (de moment només el five-guess) per resoldre la seqüència creada pel codemaker.

2.2.4 Partida

Aquesta classe serveix per identificar totes les característiques i particularitats que formen una partida.

Per a fer això, s'utilitzen atributs que formen i singularitzen cada partida en concret.

Cada partida utilitza una identifiador únic que l'identifica.

Atributs

- **private int indentificador:** Identificador únic de la partida
- **private User jugador1:** Primer usuari que jugarà aquella partida

- **private User jugador2:** Segon usuari que jugarà aquella partida
- **private int ultima_ronda_jugada:** Número de la última ronda jugada en aquella partida
- **private boolean jugador1_es_codemaker:** Booleà que permet identificar si el jugador1 és el codemaker de la partida
- **private boolean ajuda:** Booleà que indica si s'ha fet servir la ajuda o no en aquesta partida
- **private dificultat dificultat:** Tipus de dificultat de la qual és aquella partida. És constant durant tota la partida
- **private int num_colors:** Número de colors dels quals es pot compondre el codi d'aquella partida.
- **private int temps_max:** Temps màxim en el que es pot fer la partida. També depèn de la dificultat i també és constant
- **private int num_rondes_max:** Número de rondes màximes que té el codebraker per desxifrar la combinació. Depèn de la dificultat i és constant en tota la partida
- **private boolean partida_acabada:** Booleà que indica si aquella partida està acabada o no
- **private Instant temps_inici:** Moment en el qual s'ha començat la partida
- **private List<Ronda> llista_rondes:** Llista de les diferents rondes de la partida, van desde 0 fins num_rondes_maxim-1
- **private Sequencia sequencia_solucio:** Seqüència de colors intentada que simbolitza l'intent de resoldre el codi

Mètodes principals

- **public boolean temps_excedit():** Retorna true si el temps usat > temps maxm de la partida
- **public boolean temps_usat():** Retorna el temps usat de una partida
- **public void codebreaker_guanya() & public void codemaker_guanya():** aquestes funcions acaben la partida posant com a guanyador i perdedor els respectius codemaker i codebreaker

2.2.5 Ronda

Aquesta classe fa referència a una ronda que es juga a una partida determinada. Per tant, l'identificador d'aquesta ronda dins d'una partida concreta és el seu `num_ronda`.

Conseqüentment, cada ronda identificada per un `num_ronda` és única en la partida de la qual forma part.

En aquesta ronda, es “juga” amb la seqüència intentada enviada pel codebreaker i es comprova aquesta seqüència intentada amb una seqüència de verificació del codi fet pel codemaker.

Atributs

- **private int id_partid:** Identificador únic, dins de la partida, de la ronda.
- **private int num_ronda:** Número de la ronda dins de la partida
- **private Sequencia sequencia_intentada:** Seqüència de colors intentada pel codebreaker
- **private Sequencia sequencia_verificacio:** Seqüència de verificació que posa el codemaker per verificar la `sequencia_intentada`

Mètodes principals

- **public Sequencia get_seq_intentada:** Metode que retorna la Sequencia Intentada.
- **public Sequencia get_seq_verificacio:** Metode que retorna la Sequencia de Verificació.
- **public void set_intentada:** Modifica la `sequencia_intentada` de la classe per la classe Sequencia que passem per paràmetre.
- **public void set_verificacio:** Modifica la `sequencia_verificacio` de la classe per la classe Sequencia que passem per paràmetre.
- **public boolean check_sequencia_encertada:** Comprova que la `sequencia_verificacio` sigui tot negre i envia

2.2.6 Seqüència

Aquesta classe fa referència a la seqüència (tan intentada, per part del codebreaker, com de verificació, per part del codemaker). En aquesta classe es comprova si la seqüència intentada pel codebreaker és correcta en termes de correcte posicionament dels colors i sens ellocs buits i es fa el posicionament d'espigues (tant blanques com negres) per fer la comprovació

de la correctesa de la seqüència. Per exemple, a *valida_seq()* :: *bool* es comprova que estigui ben escrita (no de valor sinó de colors).

Atributs

- **private colors[] array:** Array amb les 4 posicions de les boles. Les boles poden ser de diferents colors. Les seqüències intentades poden tindre fins al color num_color-1, mentre que la seqüència de verificació pot tindre els colors blanc, negre o no tindre bola(null)
- **private type_seq tipus:** Especifica quin tipus de seqüència és amb la que s'està treballant

Mètodes principals

- **private void set_position:** Inserta un color en un posició determinada.
- **private List<Integer> toListInteger:** Retorna el array de colors en forma de llista de Integers.

2.2.7 Seqüència Intentada

Subclasse de seqüència que fa referència a l'intent de seqüència realitzat en una ronda d'una partida concreta.

Atributs

- **Tots els heredats de la superclasse Seqüència**

Mètodes principals

- **private void set_array:** Modifica el array de la classe per el que passem per paràmetre fent les comprovacions necessàries.

2.2.8 Seqüència Solució

Subclasse de seqüència que fa referència a l'intent de seqüència realitzat en una ronda d'una partida concreta.

Atributs

- **Tots els heredats de la superclasse Seqüència**

Mètodes principals

- **private void set_array:** Modifica el array de la classe per el que passem per paràmetre fent les comprovacions necessàries.

2.2.9 Seqüència Verificació

Subclasse de seqüència que fa referència a la seqüència de verificació realitzat en una ronda d'una partida concreta.

Atributs

- **Tots els heredats de la superclasse Seqüència**

Mètodes principals

- **private void set_array_verificacio:** Modifica el array de la classe per el que passem per paràmetre fent les comprovacions necessàries
- **private boolean valida_sequencia:** Comprova que la seqüència de verificació sigui correcte, fent un comprovació de la esperada i la que ens donen. En cas que siguin idèntiques retorna True, i False en cas que siguin diferents.

2.2.10 Rànkking

Aquesta classe fa referència al rànkking que se li mostrarà al usuari. Aquest rànkking, que pot ser de tipus fàcil, normal, difícil o pvsp, fent referència a al tipologia de partida que s'ha triat per jugar a l'inici, mostrarà un rànkking amb una classificació típica dels estàndards: apareixent l'usuari (amb el seu respectiu posicionament, i els seus punts en aquell tipus de rànkking). D'aquesta manera, es pot seguir de manera fàcil i àgil l'evolució dels usuaris dins del joc, dins de cada dificultat i es poden realitzar comparacions a plena vista del desenvolupament de cada usuari (incloent-hi *user_maquina* que hi participen, menys al PvsP).

Atributs

- **protected List<Pair<Pair<Double, String>, LocalDate>> rank:** Llista que conté els usuaris amb els seus respectius punts i la data d'actualització d'aquesta posició del rànkig.

Mètodes principals

- **nova_partida_ranking(double punts, String nom_user):** es comprova a quina posició entra l'usuari. S'inserta els punts de l'usuari(a canviar a winrate) juntament amb el nom usuari i la data on s'ha entrat al ranking.

2.2.11 Ranking fàcil

Subclasse de rànkig que fa referència al rànkig amb punts i usuaris amb partides jugades de tipus fàcil.

Atributs

- **Tots els heredats de la superclasse Rànkig**

2.2.12 Ranking_normal

Subclasse de rànkig que fa referència al rànkig amb punts i usuaris amb partides jugades de tipus normal.

Atributs

- **Tots els heredats de la superclasse Rànkig**

2.2.13 Ranking_difícil

Subclasse de rànkig que fa referència al rànkig amb punts i usuaris amb partides jugades de tipus difícil.

Atributs

- **Tots els heredats de la superclasse Rànkig**

2.2.14 Ranking_pvp

Subclasse de rànking que fa referència al rànking amb punts i usuaris amb partides jugades de tipus pvp.

Atributs

- **Tots els heredats de la superclasse Rànking**

2.2.15 Record

Aquesta classe fa referència a un record en concret que forma part del sistema del joc. Aquest record és complet per un i només un usuari, el qual es pot anar transformant segons les variacions i actualitzacions de punts de part d'aquests usuaris del sistema.

Atributs

- **private String nom_record:** Nom del record que l'identifica (i s'avalua)
- **private String nom_usuari:** Nom de l'usuari que ha batut el rècord anterior a el seu
- **private int punts:** Punts que l'usuari ha puntuat al batre el record.

Mètodes principals

- **check_if_record(int punts, String nom_usuari):** funció on es passen els punts que s'han fet en una partida i el nom de l'usuari que els ha aconseguit. Es comprova si supera al actual record i si el supera s'actualitza amb els paràmetres. Si no es supera es manté l'antic record.

2.2.16 Type_user

Enumeration que permet identificar la tipologia de l'usuari que fa referència a la classe User. D'aquesta manera, cada usuari serà d'un tipus o de l'altre i això ens permetrà desenvolupar diferents funcionalitats i mostrar errors en funció del tipus d'usuari que sigui.

Aquests tipus són:

1. user_persona,
2. user_maquina

2.2.17 dificultats

Enumeration que identifica el tipus de dificultat del qual és una partida en concret.

Aquets enumeration consta de diferents atributs (que es mostren a continuació) que prenen valors diversos en funció del tipus de dificultat que es tracti.

1. private String dificultat;
2. private int num_dificultat;
3. private int num_colors;
4. private int temps_max;
5. private int num_max_rondes;

- Si es tracta d'una dificultat fàcil, aquests atributs prenen els valors de:
FACIL("facil",1,4,600,14)
- Si es tracta d'una dificultat normal, aquests atributs prenen els valors de:
NORMAL("normal",2,6,600,10)
- Si es tracta d'una dificultat difícil, aquests atributs prenen els valors de:
DIFICIL("difcil",3,8,600,6)

D'aquesta manera, ens permet distingir el tipus de partida a jugar, la seva dificultat, la seva jugabilitat i tot el que implica per tema rànkings,...

2.2.18 colors

Enumeration que ens permet classificar els colors (assignant-los un id únic per a cada un).

D'aquesta manera, podríem crear seqüències d'intent i de verificació més còmodament i seria una metodologia d'identificació de les mateixes ràpida, fàcil de modificar i amb una alta usabilitat.

Aquests colors són i estan associats a:

1. NULL("null",0),
2. VERMELL("vermell",1),
3. VERD("verd",2),
4. BLAU("blau",3),
5. GROC("groc",4),
6. MAGENTA("magenta",5),
7. CIAN("cian",6),
8. MARRO("marro",7),
9. GRIS("gris",8),
10. BLANC("blanc",9),
11. NEGRE("negre",10);

2.2.19 type_seq

Enumeration que permet identificar la tipologia de seqüència que fa referència a la classe Seqüència. D'aquesta manera, cada seqüència serà d'un tipus o de l'altre i això ens permetrà desenvolupar diferents funcionalitats i mostrar errors en funció del tipus de seqüència que sigui.

Aquests tipus són:

1. solucio,
2. intentada,
3. verificacio

2.2.20 Pair

Aquesta classe fa referència al `@override` que fem a la classe ja implementada Pair.

Aquestes substitucions s'utilitzaran en molts dels atributs de les classes comentades anteriorment, com per exemple al rànquing.

Naturalment, els atributs d'aquesta classe són:

1. `private final A first;`
2. `private final B second;`

3. Estructures de dades i algorismes utilitzats

3.1 Estructures de dades

En aquesta secció es comenten les estructures de dades que hem utilitzat i perquè ho hem fet.

Dins del controlador de domini tenim guardats els usuaris, els records i els rankings en HashMaps ja que haurem de fer bastantes modificacions en aquestes estructures i els Maps son molt eficients en cerca, addició i esborrament de dades. Lo bó d'aquesta estructura de dades és que dona un cost mitjà constant de temps en les 3 operacions. Per altre banda, fem servir un array de l'enumeration *colors* per guardar les seqüències de “boletes” que s'entren, ja sigui per provar d'encertar com de verificar si l'anteriorment mencionada és correcte o no.

A controlador de partida tenim un altre HashMap per guardar les partides per la mateixa raó que fem servir-la a controlador de domini.

Dins de la classe User fem servir l'estructura ArrayList per guardar les partides que té l'usuari acabades i una altra per a guardar les partides acabades de l'usuari, això es deu a que aquesta estructura permet modificar el seu tamany, cosa ideal per anar afegint i treient partides de les llistes. També ens ofereix cost $O(1)$ per a accedir als elements, que ho fa ideal per buscar partides concretes. Com els accessos a aquesta estructura seran majoritàriament accessos aleatoris, va millor que una LinkedList (millor per implementar cues) o un TreeSet (millor per mantenir els elements ordenats i fer accessos seqüencials).

Dins de Ranking guardem una LinkedList amb el string del nom del usuari situat el rànking, els punts que té en el total de partides jugades en aquella dificultat, i una simple LocalDate que ens permet tenir constancia de quan s'ha actualitzat el seu valor. Això ens podria ser útil per mostrar el rànking d'usuaris per pantalla i fallitaria al reconeixement de cada un dels usuaris degut a la gran comoditat que resulta poder veure ordenats per punts els participants del joc (i inclús fa quant que no juguen una partida d'aquella dificultat, és a dir, fa quan no se'ls hi ha actualitzat el posicionament al rànking global d'aquella dificultat en específic).

Finalment, i parlant de les dificultats, hem utilitzat un enumeration per definir les nostres pròpies, diferenciant-les pel número de colors a endevinar, el temps màxim de partida i el número màxim de ronde per a completar-la. Això ho hem fet amb un enumeration que es redefineix amb atributs i mètodes.

3.2 Algoritmes

Five Guess Algorithm

L'algorisme Five Guess és una estratègia per resoldre el joc de Mastermind que consisteix a fer un nombre limitat d'intents basats en una selecció cuidadosa dels codis proposats, amb la finalitat de reduir ràpidament l'espai de solucions possibles fins a trobar la solució correcta. A continuació, es descriuen les funcions i estructures de dades utilitzades en l'algorisme:

- **List:** és una estructura de dades que s'utilitza per emmagatzemar una seqüència d'elements ordenats en una llista. En aquest algorisme, s'utilitza una llista d'enters per representar les seqüències de colors del codi a endevinar i dels codis proposats.
- **Arrays:** és una classe d'utilitat a Java que proporciona diversos mètodes útils per treballar amb matrius (arrays). En aquest algorisme, s'utilitza el mètode `asList()` de la classe `Arrays` per crear una llista d'enters a partir d'un array d'enters.
- **ArrayList:** és una implementació de la interfície `List` que utilitza una matriu dinàmica per emmagatzemar elements. En aquest algorisme, s'utilitza un `ArrayList` per emmagatzemar les seqüències de colors de tots els codis possibles i dels codis proposats.
- **Math.pow():** és un mètode estàtic de la classe `Math` a Java que retorna la potència d'un nombre donat. En aquest algorisme, s'utilitza `Math.pow()` per calcular el nombre de codis possibles que poden ser generats a partir d'una llista de colors i una longitud donada.
- **Random:** és una classe a Java que s'utilitza per generar nombres aleatoris. En aquest algorisme, no s'utilitza explícitament la classe `Random`, però la selecció dels codis proposats es basa en un algorisme de selecció aleatòria.
- **generateCodes():** és una funció que genera tots els codis possibles a partir d'una llista de colors i una longitud donada. Utilitza un algorisme d'enumeració exhaustiva que genera tots els codis possibles utilitzant `row nested loops` (el vem aprendre a DBD). També podria haver utilitzat una funció recursiva. En termes de eficiència els 2 algoritmes (`row nested` i `recursiu`) son iguals $O(C^n)$ on **C** es el nombre de colors i **n** el nombre de ranures (en el nostre cas 4). No obstant vem llegir que el `row nested` és més eficient ja que evita la sobrecarga recursiva i la creació d'una gran quantitat d'objectes en el procés de generació.
- **getResult():** és una funció que retorna el resultat (blanques i negres) d'una possible solució. Prendre com a entrada una seqüència de colors (intent) i la seqüència de colors correcta (solució), i retorna una llista d'enters que representa el nombre de fitxes negres i blanques obtingudes en l'intent. Aquesta funció la podriem haver

reutilitzat d'alguna funció de seqüència, però com l'enunciat deia que havíem de tenir una funció solve que retornés una llista de llistes de integers i que li arribes com a solució una llista de integers no ho vam fer (nosaltres guardem les seqüències de colors com un array de l'objecte color que és una enumeration). Per això no vem reutilitzar codi.

- `filterCodes()`: és una funció que filtra de tots els codis possibles els que tenen el mateix resultat que la seqüència de colors intentada. Prendre com a entrada una llista de codis possibles, la seqüència de colors intentada i el resultat obtingut en l'intent, i retorna una llista de codis possibles que tenen el mateix resultat que la seqüència de colors intentada.
- `minimax()`: es una funció que retorna la millor solució possible donada una llista de possibles solucions. Pren com a entrada una llista de possibles solucions, i utilitza un algoritme de minimax per a trobar la solució que minimitza el nombre màxim de possibles solucions restants.

La funció `filterCodes` s'utilitza per filtrar la llista de codis possibles i eliminar aquells que no compleixen amb les condicions establertes. Rep una llista de possibles codis, la seqüència intentada i el resultat de l'intentada i retorna una llista de codis possibles filtrats que compleixen amb el resultat de la seqüència intentada.

La funció `minimax` s'utilitza per trobar la millor solució possible donada una llista de possibles solucions. Aquesta funció avalua cada possible codi i calcula quants codis possibles són consistents amb cada possible codi. Llavors, selecciona el codi que té la menor quantitat de codis possibles consistents amb ell. D'aquesta manera, es tracta de minimitzar el nombre màxim de possibilitats que podrien resultar del següent intent.

Aquestes dues funcions són essencials en l'algoritme Five Guess perquè ajuden a reduir el nombre d'intents necessaris per trobar la solució. Filtrar els possibles codis basant-se en els resultats de les jugades anteriors ajuda a reduir significativament l'espai de cerca. L'ús de la funció `minimax` permet escollir la millor solució possible de manera més eficient i reduir el nombre d'intents necessaris per arribar a la solució final.

4. Relació de les classes implementades per membre de l'equip

En aquest últim apartat de la primera entrega del projecte, es mostra la repartició de classes que s'ha portat a terme entre els membre del grup, el perquè d'aquesta repartició i quins han sigut els punts forts i febles d'aquesta decisió presa.

Cada classe ha sigut desenvolupada per un integrant però, com és obvi en un treball en grup, cadascú donava la seva opinió de com creia que s'havia de fer, quina estructura era la millor, i comentava quins canvis calia fer per tal de que aquestes classes es relacionessin el millor possible amb les altres implementades.

JORDI BARANDA	JUAN CLUSELLAS	FERRAN SOLANES	MARC TURU
Seqüència + Test	Partida + Test	Record + Test	User + Test
Ronda + Test	Five Guess	Ranking + Test	User Persona + Test
Driver Domini		Controlador Partida	User Maquina + Test
Makefile		<i>Pair</i>	Controlador Domini
<i>Type Seq</i>			<i>Type user</i>
<i>Colors</i>			
<i>Dificultats</i>			